BEHIH Maroua
Étudiante en L3 informatique – Université d'Angers

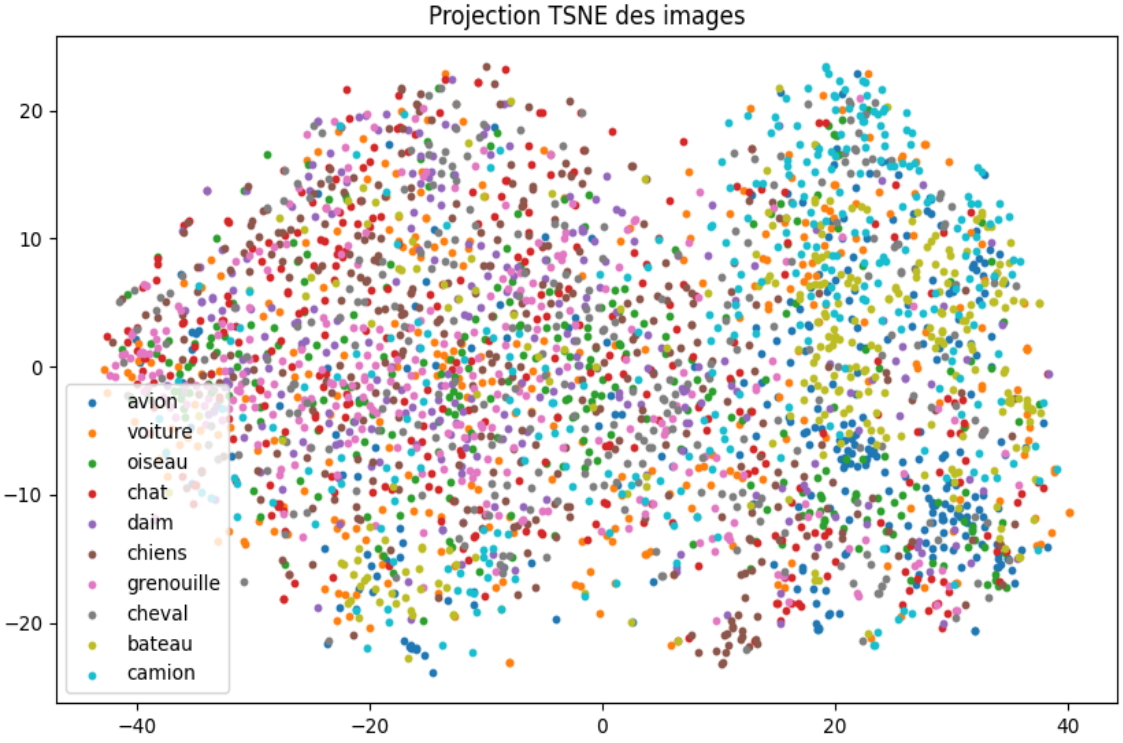
Rapport de Projet : Classification multi-classes d'images

1 Prise au main de la plate-forme de challenge Codabench

- Après soumission du fichier images_test_predictions.csv, j'ai obtenu une précision de 0.1041, soit environ 10.41 %.
- Ce résultat est très proche de 10 %, ce qui correspond à la précision attendue pour une classification réalisée aléatoirement dans un problème comportant 10 classes.
- Je suppose donc que le fichier images_test_predictions.csv a été généré en attribuant une classe aléatoire parmi les 10 possibles à chaque image du dataset de test.

2 Phase d'entraînement et de validation

2.1. Visualisations des données :



- Les 10 classes ne sont pas nettement isolées. On observe plusieurs amas de points correspondant aux différentes classes, mais ces groupes ne sont pas totalement séparés et présentent un certain chevauchement, notamment au centre du graphique.

- Certaines classes semblent former des groupes relativement homogènes (par exemple les avions ou les bateaux). En revanche, d'autres classes se chevauchent davantage en raison de similarités visuelles entre certaines catégories comme chat / chien ou voiture / camion, ce qui rend la frontière de décision plus difficile pour un algorithme basé uniquement sur les pixels bruts.

2.2. K-Nearest Neighbors (KNN)

Paramètre principal :

Le principal paramètre du modèle KNN est :

k : nombre de voisins considérés pour effectuer la prédiction.

Présentation des résultats :

- Expérience 1 :
  - Intervalle testé : $k \in \{1, 3, 5, ..., 29\}$
  - Résultats :
    - Meilleur k = **5** → score = **0.1932**
    - Meilleur k = **27** → score = **0.2177**
- Expérience 2 :
  - Intervalle testé : $k \in \{1, 3, 5, ..., 49\}$
  - Résultats :
    - Meilleur k = **45** → score = **0.2046**

Analyse des résultats :

- Globalement, les performances du KNN restent faibles, avec une précision autour de **20%**. On observe que les meilleurs résultats sont obtenus pour des valeurs relativement élevées de k, ce qui permet de réduire l'effet du bruit. En effet, un k trop petit rend le modèle très sensible aux variations locales (sur-apprentissage), tandis qu'un k trop grand lisse excessivement les frontières de décision (sous-apprentissage).

- La variation du meilleur k d'une expérience à une autre s'explique principalement par :
  - la randomisation du découpage (train/validation)
  - la taille réduite du dataset (4000 images utilisées)

Limites du KNN :

Le KNN présente plusieurs limitations importantes pour la classification d'images.

- Chaque image est représentée par un vecteur de 3072 dimensions, ce qui entraîne le phénomène du *curse of dimensionality*. Dans un espace de grande dimension, les distances deviennent moins discriminantes, ce qui réduit fortement l'efficacité du modèle.

- Le modèle compare directement les valeurs des pixels sans prendre en compte la structure spatiale de l'image. Ainsi, un léger déplacement d'un objet (par exemple un chat légèrement décalé) peut être interprété comme une différence importante, ce qui dégrade la qualité des prédictions.

- De plus, le KNN ne capture pas les caractéristiques essentielles des images telles que les formes, les textures et les contours.

- Enfin, le KNN présente également des limites pratiques :
  - **Mémoire :** il nécessite de stocker l'ensemble des données d'entraînement
  - **Temps de calcul :** chaque prédiction implique le calcul des distances avec tous les exemples
  - **Pas d'apprentissage :** il s'agit d'un modèle dit *lazy*, sans phase d'entraînement explicite

Conclusion :

En conclusion, le KNN donne des performances limitées sur ce problème de classification d'images, avec une précision d'environ 22%. Malgré l'ajustement du paramètre k, cette méthode reste peu adaptée à ce type de données. Des modèles plus avancés, capables d'extraire des caractéristiques pertinentes, sont nécessaires pour améliorer significativement les résultats.

2.3. Régression Logistique Multivariée

Paramètres du modèle :

La régression logistique multiclasse repose sur plusieurs paramètres importants :

- **nb_epochs** : nombre d'itérations d'entraînement
- **learning rate (lr)** : taux d'apprentissage
- **optimiseur** : Adam

Présentation des résultats :

Plusieurs combinaisons de paramètres ont été testées afin d'évaluer leur impact sur les performances

Nb Époques	Learning Rate (lr)	Validation Accuracy	Score Challenge
100	**0.001**	**0.39675**	**0.395**
5000	0.01	0.374	0.379
5000	0.001	0.3885	0.381
8000	0.001	0.377	0.375

Analyse des résultats :

La régression logistique offre une amélioration significative par rapport au KNN, avec une précision proche de **40%**. On observe que **l'augmentation du nombre d'époques** n'améliore pas nécessairement les performances. Au contraire, les résultats montrent qu'un nombre trop élevé d'itérations peut dégrader la précision. Cela peut s'expliquer par un phénomène de **sur-apprentissage (overfitting).**

Concernant le **taux d'apprentissage**, on observe qu'un learning rate trop élevé (0.01) empêche une convergence stable et conduit à des performances plus faibles. À l'inverse, un learning rate plus faible (0.001) permet une convergence plus progressive et plus stable, ce qui se traduit par de meilleures performances.

Interprétation :

Contrairement au KNN, qui se contente de stocker l'ensemble du dataset, la régression logistique apprend un poids ($\boldsymbol{w}$) pour chaque pixel et pour chaque classe. Le modèle construit ainsi une représentation globale des données en attribuant une importance différente à chaque pixel.

On peut interpréter ce mécanisme comme la création de « filtres moyens » pour chaque classe (par exemple, un filtre qui détecte certaines couleurs ou motifs caractéristiques d'une classe donnée). Ainsi, le modèle est capable de pondérer l'importance de certaines zones de l'image, ce que le KNN ne peut pas faire.

Limites de la régression logistique :

Malgré ses meilleures performances, la régression logistique présente plusieurs limitations importantes.

- **Linéarité :** la régression logistique est un modèle linéaire. Elle suppose qu'une classe peut être séparée des autres par un hyperplan. Or, dans le cas des images, les différences entre classes (par exemple entre un chat et un chien) sont fortement non linéaires, ce qui limite les performances du modèle.
- **Perte de structure spatiale :** en transformant l'image en un vecteur de 3072 valeurs (flatten), le modèle perd totalement la notion de voisinage entre les pixels. Il ne peut donc pas exploiter la structure spatiale de l'image.
- **Sensibilité aux transformations** : comme pour le KNN, un léger décalage ou une rotation de l'objet peut perturber fortement la prédiction, car les poids appris ne correspondent plus aux bonnes positions dans l'image.

Conclusion :

La régression logistique double l'efficacité du KNN en apprenant des représentations globales des classes. Cependant, son incapacité à modéliser des relations non-linéaires et à préserver la géométrie spatiale des images explique pourquoi nous devons nous tourner vers les réseaux de neurones, capables d'apprendre des représentations non linéaires et de mieux exploiter la structure des images pour dépasser le palier des 40 % de précision.

2.4. Réseau de Neurones (couches linéaires)

Paramètres du modèle :

Le réseau de neurones utilisé est un modèle entièrement connecté composé de plusieurs couches linéaires avec fonctions d'activation.

Les principaux paramètres étudiés sont :

- **nb_epochs** : nombre d'itérations d'entraînement
- **learning rate (lr)** : taux d'apprentissage
- **architecture du réseau** : nombre de couches et nombre de neurones (h1, h2, h3)
- **fonction d'activation** : Sigmoid puis ReLU
- **batch_size** : taille des mini-batchs (via DataLoader)
- **optimiseur** : Adam

Présentation des résultats :

Plusieurs configurations ont été testées afin d'évaluer l'impact des différents paramètres.

Nb Époques	Learning Rate (lr)	Architecture	Activation	Batch_size	Validation Accuracy	Score Challenge
100	0.001	512,256	Sigmoid	—	0,456	0.4318
100	0.001	200,100	ReLU	—	0.46775	0.4716
100	0.001	300,150	ReLU	—	0.4815	0.4648
100	0.001	400,200	ReLU	—	0.479	0.4647
100	0.001	512,256,128	ReLU	—	0.441	0.4392
150	0.001	512,256,128	ReLU	—	0.479	0.4705
5000	0.001	512,256,128	ReLU	—	0.462	0.4543
100	0.001	1024,512	ReLU	—	0.48525	0.4748
150	0.0005	512,256,128	ReLU	64	0.4875	0.4833
150	**0.005**	**512,256,128**	**ReLU**	**128**	**0.4975**	**0.4952**

Analyse des résultats :

- Un point important observé est l'impact de la fonction d'activation. Le passage de **Sigmoid** à **ReLU** a permis une amélioration notable des performances. En effet, la fonction ReLU évite les problèmes de saturation du gradient et favorise un apprentissage plus rapide et plus efficace.
- **L'architecture du réseau** joue également un rôle déterminant. Des architectures plus larges, comme (1024, 512), permettent d'obtenir de meilleurs résultats en augmentant la capacité du modèle. En revanche, l'ajout de couches supplémentaires, comme dans (512, 256, 128), ne garantit pas une amélioration systématique et peut même dégrader les performances si les hyperparamètres ne sont pas correctement ajustés.
- Concernant le **nombre d'époques**, les résultats montrent qu'une augmentation importante (jusqu'à 5000 itérations) n'apporte pas de gain significatif, et peut même entraîner une dégradation des performances. Cela suggère un phénomène de sur-apprentissage ou une convergence vers un minimum peu optimal.
- L'introduction du **DataLoader avec mini-batchs** constitue une amélioration importante. L'utilisation d'un *batch size* (64 ou 128) permet une convergence plus stable, une meilleure généralisation du modèle, et contribue à améliorer les performances globales, atteignant ainsi environ **49% de précision**.

Interprétation :

Contrairement à la régression logistique, ce modèle introduit des couches cachées non linéaires, ce qui lui permet de modéliser des relations plus complexes entre les pixels.

Chaque couche du réseau apprend une transformation des données, permettant progressivement d'extraire des représentations plus abstraites. Le modèle est donc capable de combiner les pixels pour détecter des motifs plus complexes. Cependant, comme les images sont toujours aplaties (flatten), le réseau ne tient pas compte de la structure spatiale des données.

Limites :

Malgré de meilleures performances, ce modèle présente encore plusieurs limites.

- **Perte de structure spatiale :**
  - Les images sont transformées en vecteurs
  - Aucune notion de voisinage entre pixels
- **Nombre important de paramètres :**
  - Risque de sur-apprentissage
  - Besoin de réglage fin des hyperparamètres
- **Sensibilité aux transformations :** translations ou rotations non prises en compte

Conclusion :

Le réseau de neurones à couches linéaires améliore les performances par rapport à la régression logistique, en atteignant une précision proche de **50%** grâce à l'introduction de non-linéarités.
Cependant, l'absence de prise en compte de la structure spatiale des images constitue une limite majeure. Pour dépasser ce niveau de performance, il est nécessaire d'utiliser des architectures spécialisées comme les réseaux de neurones convolutifs (CNN), capables d'exploiter la structure des images.

2.5. Réseau de convolution (CNN)

Paramètres du modèle :

Les principaux paramètres et composants étudiés sont :

- **nb_epochs** : nombre d'itérations d'entraînement
- **learning rate (lr)** : taux d'apprentissage
- **Architecture Convolutive :** Utilisation de 3 couches de convolution (Conv2d) avec des filtres augmentant progressivement (32, 64, 128).
- **Batch Normalization (BatchNorm2d) :** Ajoutée après chaque convolution pour stabiliser et accélérer l'apprentissage.
- **Batch size** : taille des mini-batchs (via DataLoader)
- **fonction d'activation** : ReLU
- **optimiseur** : Adam avec un Scheduler (StepLR) pour réduire le taux d'apprentissage en cours de route.

Présentation des résultats :

Les tests effectués montrent une progression constante de la précision en fonction de l'ajustement de la taille des batchs et du nombre d'époques.

Nb Époques	Learning Rate (lr)	Batch_size	Validation Accuracy	Score Challenge
10	0.001	128	0.63	0.6287
40	0.0003	64	0.69775	0.6936
50	0.0005	64	0.724	0.7202
60	**0.0003**	**32**	**0.73025**	**0.7346**

Analyse des résultats :

- Efficacité des convolutions : Dès les 10 premières époques, le CNN surpasse largement le meilleur score du MLP (environ 50%). Cela démontre que les filtres convolutifs parviennent à capturer des motifs visuels (bords, textures, formes) que les couches linéaires ne voyaient pas.
- Impact de la taille du Batch : On observe qu'une réduction de la taille du batch (passage de 128 à 32) combinée à un nombre d'époques plus élevé a permis d'affiner l'apprentissage et d'atteindre le meilleur score de 73.46%.
- Stabilité grâce à la Batch Normalization : L'intégration des couches de normalisation a permis d'utiliser des taux d'apprentissage plus variés sans que le modèle ne diverge, rendant l'entraînement beaucoup plus robuste.
- Gestion de l'Overfitting : L'utilisation du Dropout à 0.5 et du weight_decay dans l'optimiseur a permis de maintenir une cohérence étroite entre l'accuracy de validation et le score réel sur le jeu de test.

Interprétation :

Contrairement aux modèles précédents, le CNN exploite la structure spatiale de l'image. Grâce aux noyaux de convolution, le réseau analyse des voisinages de pixels, ce qui lui permet de reconnaître un objet quel que soit son emplacement dans l'image (principe d'invariance par translation). Les couches de Pooling permettent ensuite de résumer cette information, rendant le modèle plus "intelligent" face aux variations visuelles.

Limites :

Bien que les performances soient excellentes (73%), certaines limites subsistent :

- **Complexité de calcul :** L'entraînement est beaucoup plus exigeant pour le processeur (CPU) que les modèles linéaires.
- **Besoin d'architecture plus profonde :** Pour dépasser les 90%, une architecture "simple" comme celle-ci ne suffit plus ; il devient nécessaire d'utiliser des architectures résiduelles.

Conclusion :

Le CNN constitue une architecture particulièrement adaptée au traitement d'images. En préservant la géométrie des données et en apprenant des représentations hiérarchiques de caractéristiques, il permet d'atteindre des performances nettement supérieures aux méthodes classiques étudiées précédemment. Ces résultats justifient l'utilisation de modèles plus avancés, tels que ResNet, afin d'exploiter des architectures plus profondes et d'améliorer encore les performances.

2.6. Architecture Résiduelle (ResNet-18)

Paramètres du modèle :

- Le modèle utilisé est une architecture ResNet18, basée sur des blocs résiduels permettant de faciliter l'entraînement de réseaux profonds. Contrairement aux CNN classiques, les connexions de raccourci permettent de mieux propager le gradient et d'éviter les problèmes de dégradation des performances lorsque la profondeur augmente.
- Les principaux paramètres utilisés sont les suivants :

- **Architecture :** ResNet18 (4 blocs résiduels)
- **Optimiseur :** SGD
- **Learning rate initial :** 0.1
- **Momentum :** 0.9
- **Weight decay :** 5e-4
- **Scheduler :** CosineAnnealingLR (T_max = 200)
- **Nombre d'époques :** 200
- **Batch size :** 128
- **Fonction de perte :** CrossEntropyLoss avec label smoothing (0.1)

- Concernant le choix de l'optimiseur, l'utilisation de **SGD** a été privilégiée, celui-ci étant généralement plus performant que Adam pour les architectures profondes comme ResNet.

- Le modèle intègre également des techniques avancées pour améliorer la performance :

- **Data augmentation (train)** : RandomCrop, RandomHorizontalFlip, AutoAugment
- **Normalisation des données**
- **Test Time Augmentation (TTA)** pour améliorer les prédictions finales

Présentation des résultats :

En raison des limitations matérielles (CPU/GPU), un seul entraînement complet a pu être réalisé avec ces paramètres. Ce modèle a permis d'obtenir :

- **Validation accuracy : 0.9280**
- **Score final : 0.941**

Ces résultats représentent une amélioration très significative par rapport aux modèles précédents.

Analyse des résultats :

- Les performances élevées obtenues s'expliquent par plusieurs facteurs. Tout d'abord, l'architecture ResNet permet d'entraîner un réseau profond sans dégradation des performances grâce aux connexions résiduelles, qui facilitent la propagation du gradient.
- L'utilisation de **data augmentation** joue également un rôle clé. En générant des variations des images (translations, flips, changements de luminosité), le modèle apprend à généraliser davantage et devient plus robuste aux variations des données.
- Le choix de **SGD avec momentum**, combiné à un scheduler de type CosineAnnealing, permet une optimisation plus stable et une meilleure convergence vers un minimum global performant.
- L'utilisation du **Test Time Augmentation (TTA)** permet d'améliorer les prédictions en moyennant plusieurs transformations d'une même image, ce qui réduit la variance et améliore la robustesse du modèle.

Interprétation :

- Le modèle ResNet18 va encore plus loin que le CNN classique en apprenant des **représentations profondes et hiérarchiques** des images. Les blocs résiduels permettent au réseau de conserver l'information importante tout en ajoutant de nouvelles transformations, ce qui améliore fortement la capacité de généralisation.
- Contrairement aux modèles précédents :
    - le CNN capturait déjà des motifs locaux
    - **ResNet capture des représentations beaucoup plus abstraites et profondes**
- Le modèle devient ainsi capable de reconnaître des objets dans des conditions variées, avec une robustesse bien supérieure aux approches précédentes.

Limites :

Malgré ses excellentes performances, ce modèle présente certaines limites.

- Tout d'abord, il nécessite des **ressources computationnelles importantes**, ce qui limite le nombre d'expérimentations possibles, comme observé dans notre cas. Le temps d'entraînement est également beaucoup plus élevé que pour les modèles précédents.
- De plus, les performances dépendent fortement du choix des hyperparamètres et des techniques d'augmentation de données. Sans ces optimisations, les résultats seraient nettement inférieurs.
- Enfin, le modèle reste complexe et moins interprétable que les modèles simples comme la régression logistique.

Conclusion :

L'utilisation de ResNet18 a permis d'atteindre des performances très élevées, avec une précision supérieure à **92 %**, marquant une amélioration majeure par rapport aux modèles précédents. Cette architecture démontre l'efficacité des réseaux profonds avec connexions résiduelles pour la classification d'images complexes.
Ces résultats mettent en évidence l'importance des techniques avancées telles que la data augmentation, le choix de l'optimiseur et les stratégies d'inférence comme le TTA. Ils confirment que les architectures profondes modernes constituent aujourd'hui l'état de l'art pour ce type de problème.